

EE115B *Digital Circuits*

Instructor: Chenxi Xiao

Part of slides are from

© Pearson Education

Prof. Hengzhao Yang

Prof. Zhifeng Zhu

Prof. Yajun Ha



上海科技大学
ShanghaiTech University

Logic Gates

Logic Gates

- A logic gate is a small transistor circuit
- Digital electronics relies on the actions of just seven types of logic gates
- The three fundamental logic gates in a digital circuit are AND, OR, and NOT

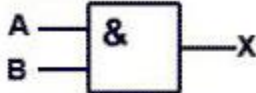
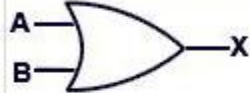
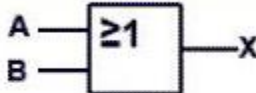
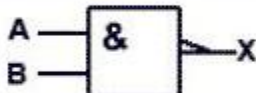
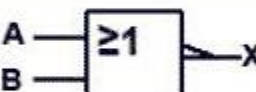
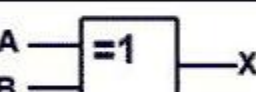
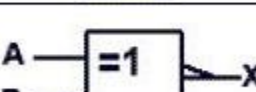
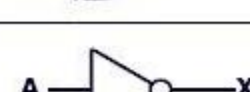
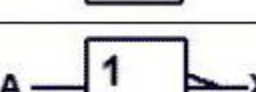
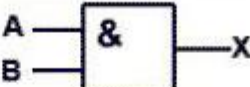
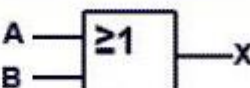
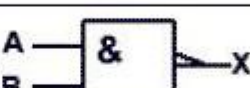
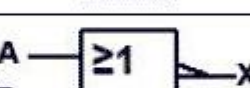
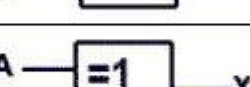
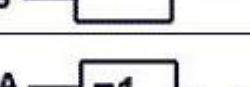
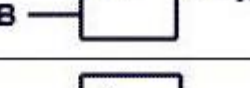
ANSI Symbol	IEC Symbol	NAME
		AND
		OR
		NAND
		NOR
		XOR
		XNOR
		NOT

Fig. 2.1.1 ANSI & IEC Gate Symbols

Boolean Statements

The actions of any of these gates can also be described using Boolean statements.

ANSI Symbol	IEC Symbol	NAME
		AND
		OR
		NAND
		NOR
		XOR
		XNOR
		NOT

Boolean	NAME
$X = A \cdot B$	AND
$X = A + B$	OR
$X = \overline{A \cdot B}$	NAND
$X = \overline{A + B}$	NOR
$X = A \oplus B$	XOR
$X = \overline{A \oplus B}$	XNOR
$X = \overline{A}$	NOT

Fig. 2.1.1 ANSI & IEC Gate Symbols



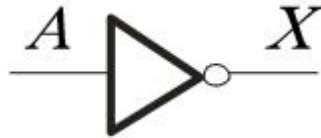
In Boolean algebra, a **variable** is a symbol used to represent an action, a condition, or data. A single variable can only have a value of 1 or 0.

The **complement** represents the inverse of a variable and is indicated with an overbar. Thus, the complement of A is $\bar{A}$.

A **literal** is a variable or its complement.



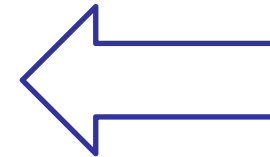
The Inverter (1)



The inverter performs the Boolean **NOT** operation. When the input is LOW, the output is HIGH; when the input is HIGH, the output is LOW.

Input	Output
A	X
LOW (0)	HIGH (1)
HIGH (1)	LOW (0)

Truth Table

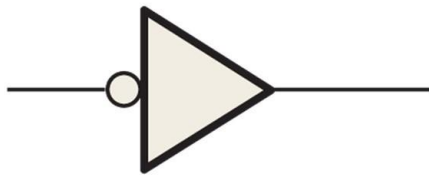
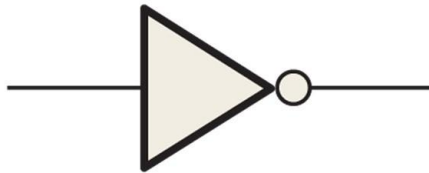


The **NOT** operation (complement) is shown with an overbar. Thus, the Boolean expression for an inverter is $X = \overline{A}$.

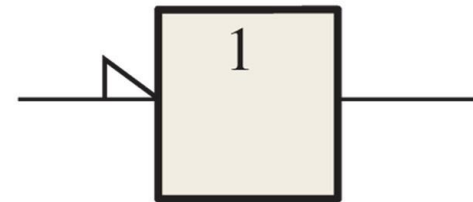
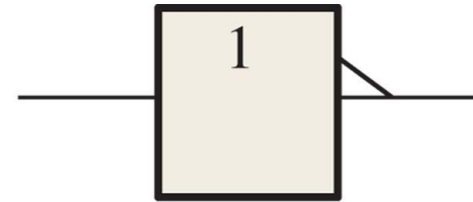


The Inverter (2)

Standard logic symbols for the inverter
(ANSI/IEEE Std. 91-1984/Std. 91a-1991).



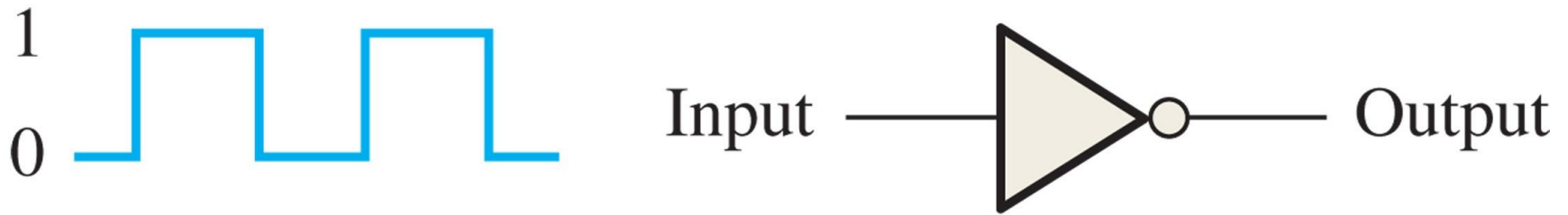
(a) Distinctive shape symbols
with negation indicators



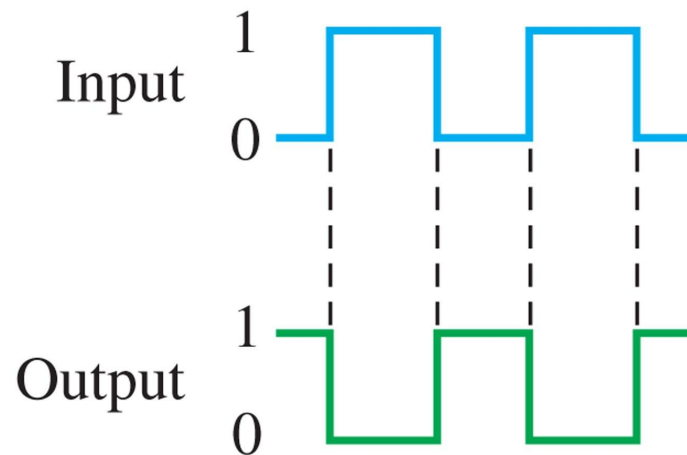
(b) Rectangular outline symbols
with polarity indicators



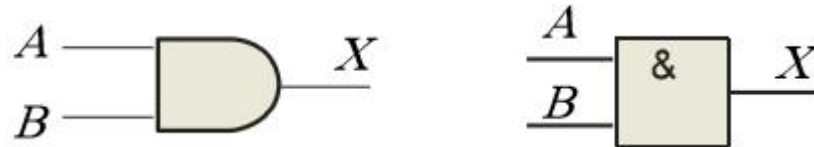
The Inverter (3)



Solution: See below for the timing diagram.



The AND Gate (1)

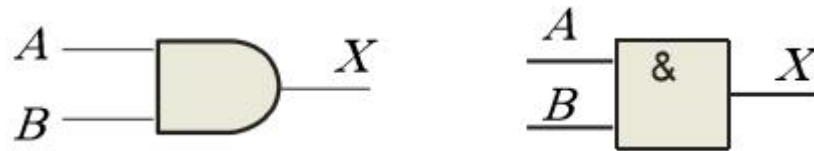


The **AND gate** produces a HIGH output when all inputs are HIGH; otherwise, the output is LOW. For a 2-input gate, the truth table is

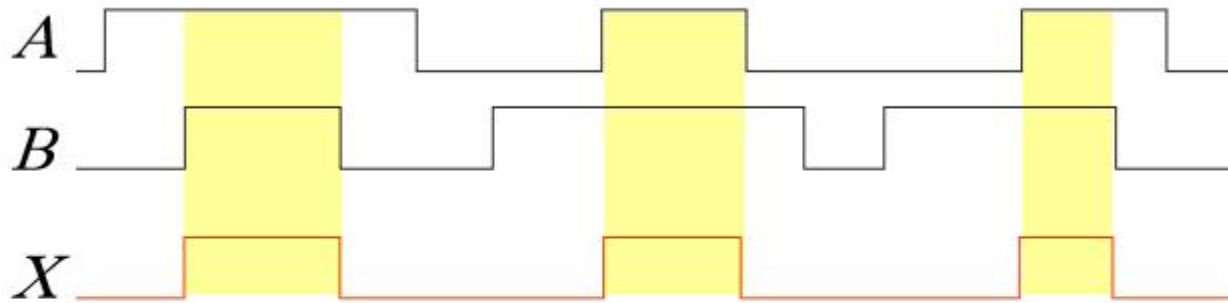
Inputs		Output
A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

The **AND** operation is usually shown with a dot between the variables but it may be implied (no dot). Thus, the AND operation is written as $X = A \cdot B$ or $X = AB$.

The AND Gate (2)



Example waveforms:

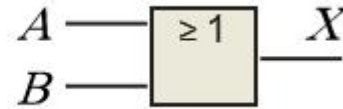


The AND operation is used in computer programming as a selective mask. If you want to retain certain bits of a binary number but reset the other bits to 0, you could set a mask with 1's in the position of the retained bits.

Example If the binary number 10100011 is ANDed with the mask 00001111, what is the result? **00000011**



The OR Gate (1)



The **OR gate** produces a HIGH output if any input is HIGH; if all inputs are LOW, the output is LOW. For a 2-input gate, the truth table is

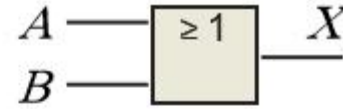
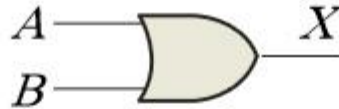
Inputs		Output
<i>A</i>	<i>B</i>	<i>X</i>
0	0	0
0	1	1
1	0	1
1	1	1

The **OR** operation is shown with a plus sign (+) between the variables. Thus, the OR operation is written as $X = A + B$.

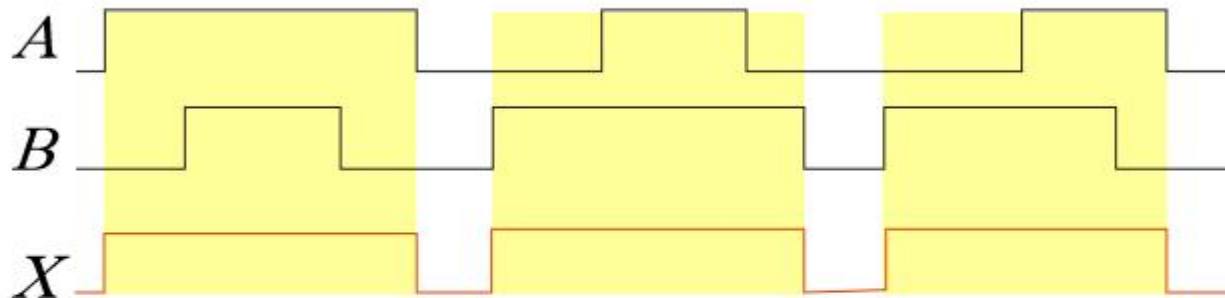


The OR Gate (2)

The OR Gate



Example waveforms:



The OR operation can be used in computer programming to set certain bits of a binary number to 1.

Example ASCII letters have a 1 in the bit 5 position for lower case letters and a 0 in this position for capitals. (Bit positions are numbered from right to left starting with 0.) What will be the result if you OR an ASCII letter with the 8-bit mask 00100000?

Solution The resulting letter will be lower case.



The NAND Gate (1)



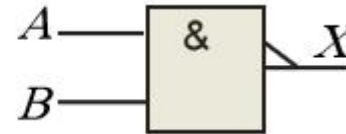
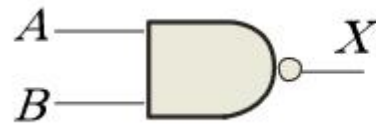
The **NAND gate** produces a LOW output when all inputs are HIGH; otherwise, the output is HIGH. For a 2-input gate, the truth table is

Inputs		Output
<i>A</i>	<i>B</i>	<i>X</i>
0	0	1
0	1	1
1	0	1
1	1	0

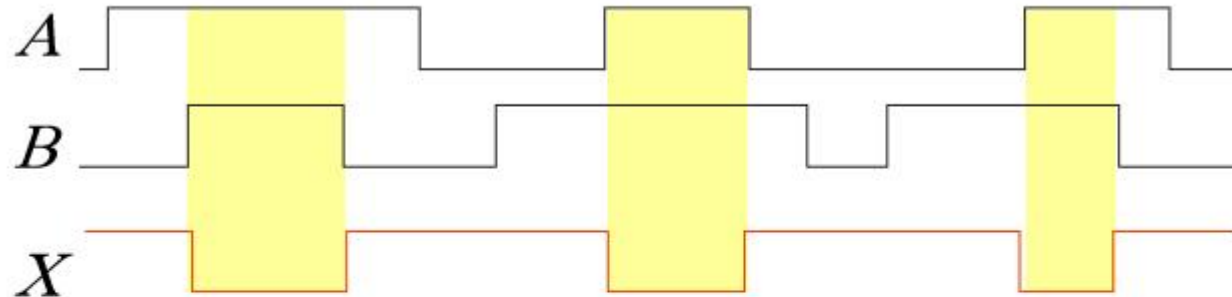
The **NAND** operation is shown with a dot between the variables and an overbar covering them. Thus, the NAND operation is written as $X = \overline{A \cdot B}$ (Alternatively, $X = \overline{A} \overline{B}$.)



The NAND Gate (2)

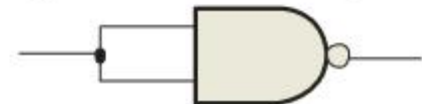


Example waveforms:

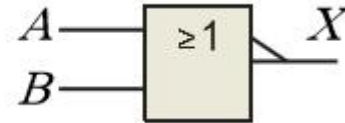
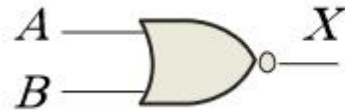


The NAND gate is particularly useful because it is a “universal” gate – all other basic gates can be constructed from NAND gates.

Question How would you connect a 2-input NAND gate to form a basic inverter?



The NOR Gate (1)



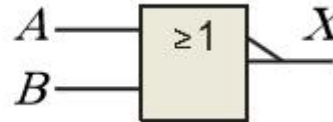
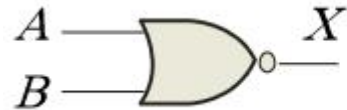
The **NOR gate** produces a LOW output if any input is HIGH; if all inputs are HIGH, the output is LOW. For a 2-input gate, the truth table is

Inputs		Output
<i>A</i>	<i>B</i>	<i>X</i>
0	0	1
0	1	0
1	0	0
1	1	0

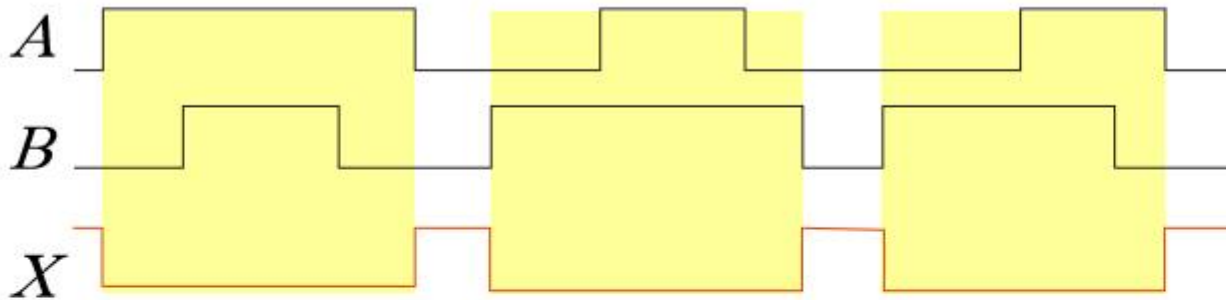
The **NOR** operation is shown with a plus sign (+) between the variables and an overbar covering them. Thus, the NOR operation is written as $X = \overline{A + B}$.



The NOR Gate (2)



Example waveforms:



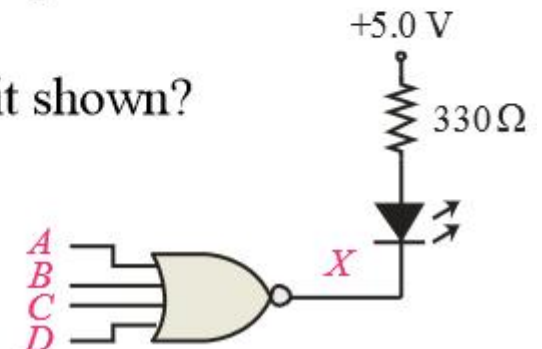
The NOR operation will produce a LOW if any input is HIGH.

Example

When is the LED is ON for the circuit shown?

Solution

The LED will be on when any of the four inputs are HIGH.



The XOR Gate (1)



The **XOR** gate produces a HIGH output only when both inputs are at opposite logic levels. The truth table is

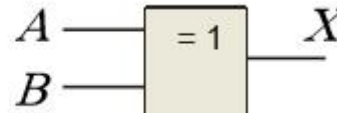
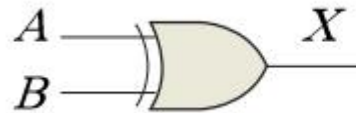
Inputs		Output
<i>A</i>	<i>B</i>	<i>X</i>
0	0	0
0	1	1
1	0	1
1	1	0

The **XOR** operation is written as $X = \bar{A}B + A\bar{B}$.

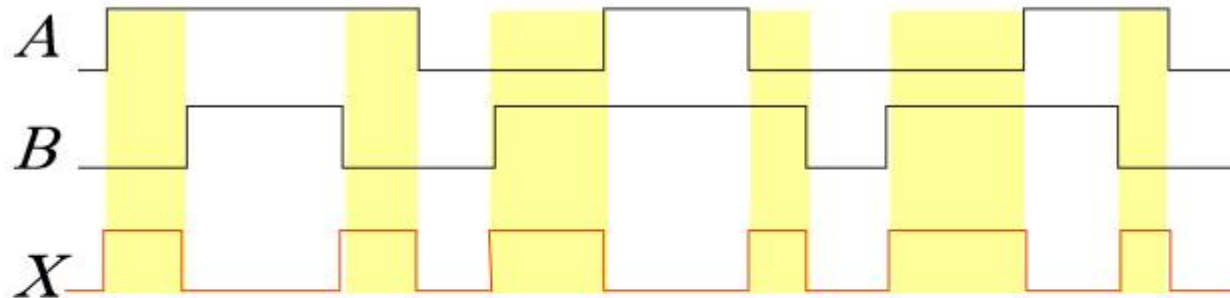
Alternatively, it can be written with a circled plus sign between the variables as $X = A \oplus B$.



The XOR Gate (2)



Example waveforms:



Notice that the XOR gate will produce a HIGH only when exactly one input is HIGH.

Question If the A and B waveforms are both inverted for the above waveforms, how is the output affected?

There is no change in the output.



The XNOR Gate (1)



The **XNOR** gate produces a HIGH output only when both inputs are at the same logic level. The truth table is

Inputs		Output
<i>A</i>	<i>B</i>	<i>X</i>
0	0	1
0	1	0
1	0	0
1	1	1

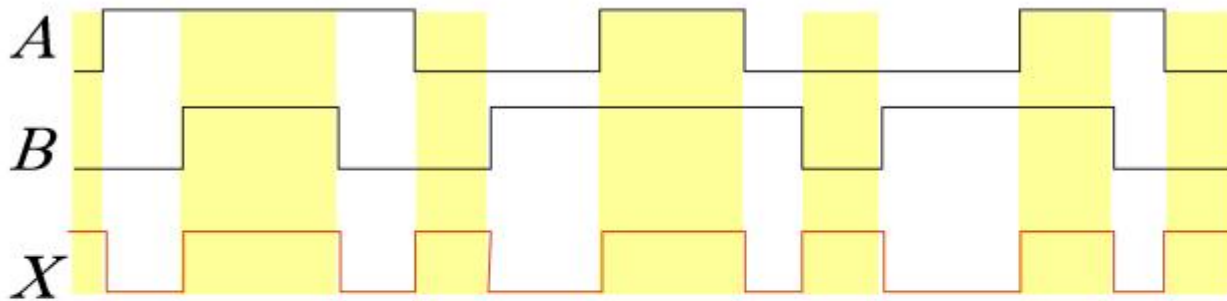
The **XNOR** operation shown as $X = \bar{A}\bar{B} + AB$. Alternatively, the XNOR operation can be shown with a circled dot between the variables. Thus, it can be shown as $X = A \odot B$.



The XNOR Gate (2)



Example waveforms:



Notice that the XNOR gate will produce a HIGH when both inputs are the same. This makes it useful for comparison functions.

Question If the A waveform is inverted but B remains the same, how is the output affected?

The output will be inverted.



Exercise 1

Determine the values of A , B , and C that make the sum term of the expression $\overline{A} + B + \overline{C} = 0$?



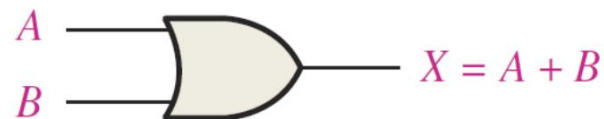
Exercise 1

Addition is equivalent to the OR operation. The sum term is 1 if one or more of the literals are 1. The sum term is zero only if each literal is 0.

Example Determine the values of $\overline{A}$, $\overline{B}$, and $\overline{C}$ that make the sum term of the expression $\overline{A} + \overline{B} + \overline{C} = 0$?

Solution Each literal must be 0; therefore $A = 1$, $B = 0$ and $C = 1$.

- Logic circuit



- Boolean expression $X = A + B$
- Truth table

A	B	$A + B = X$
0	0	$0 + 0 = 0$
0	1	$0 + 1 = 1$
1	0	$1 + 0 = 1$
1	1	$1 + 1 = 1$

Exercise 2

What are the values of the A , B and C if the product term of $A \cdot \overline{B} \cdot \overline{C} = 1$?



Exercise 2

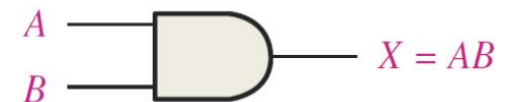
In Boolean algebra, multiplication is equivalent to the AND operation. The product of literals forms a product term. The product term will be 1 only if all of the literals are 1.

Example What are the values of the A , B and C if the product term of $A \cdot \overline{B} \cdot \overline{C} = 1$?

Solution Each literal must = 1; therefore $A = 1$, $B = 0$ and $C = 0$.

- Boolean expression $X = AB$
- Logic circuit
- Truth table

Inputs		Output
A	B	X
0	0	0
0	1	0
1	0	0
1	1	1



Operators in C

Operators	Description	Example	Result
	Bitwise OR	4 2	6
&	Bitwise AND	1 & 3	1
^	Bitwise exclusive OR	4 ^ 3	7
<<	Bitwise Left Shift	2 << 1	4
>>	Bitwise Right Shift	4 >> 1	2
~	Bitwise Not	~6	-7

Bit Manipulation in C

MASKING WITH BITWISE & :-

```
int value = 0x04030201;           // 32 bit Number

// Extracting each Byte from value from MSB to LSB
int a = (value >> 24) & 0xff;    // 04 (bits 24-31) MSB
int b = (value >> 16) & 0xff;    // 03 (bits 16-23)
int c = (value >> 8) & 0xff;     // 02 (bits 8-15)
int d = value & 0xff;           // 01 (bits 0-7) LSB

printf("\na = %x", a);
printf("\nb = %x", b);
printf("\nc = %x", c);
printf("\nd = %x", d);
```

How to set/clear/reverse bits in C?